

CRP4680/5680 Introduction to Urban Data Science

Spatial Data Analysis
Map APIs

Wenzheng Li



GeoPandas

1

Geopandas

Geopandas is a python library that allows us to manage, analyze, and map geospatial **vector** data. It combines what we have learned the tabular data analysis tools in **Pandas** with what will learn the geometry handling of **shapely** (a library for creating, analyzing, and manipulating geometric objects). Geopandas incorporates a python library called **fiona**, which handles all different kinds of spatial file formats, and **pyproj**, which manages our coordinate reference systems.

The main data structures in Geopandas are `GeoDataFrames` and `GeoSeries`, which are intended to mirror the Pandas DataFrame and Series structures. This suggests that **most functionalities of pandas also work in geopandas**.



Geopandas

A GeoDataFrame incorporates three required elements:

- a **DataFrame (df)**;
- a **geometry list**: a shapely geometry list which defines the geometric object types of each observation, e.g., points, lines, or polygons. this is the column that makes GeoDataFrame different from DataFrame.

```
[27]: data.head()
```

```
[27]:
```

	GROUP	CLASS	geometry	area
0	64	32421	POLYGON ((379394.248 6689991.936, 379389.790 6...	76.027392
1	64	32421	POLYGON ((378980.811 6689359.377, 378983.401 6...	2652.054186
2	64	32421	POLYGON ((378804.766 6689256.471, 378817.107 6...	3185.649995
3	64	32421	POLYGON ((379229.695 6685025.111, 379233.366 6...	13075.165279
4	64	32421	POLYGON ((379825.199 6685096.247, 379829.651 6...	3980.682621

(From Automating GIS Processes)

- a **crs** (coordinate reference system presented by EPSG code, e.g., "epsg: 4326");



Geopandas

Dist2CBD	DistName_En	SubwayDum	wgs_lng	wgs_lat
9345.20091	Xicheng	Level 2	116.389876	39.839173
9345.20091	Xicheng	Level 2	116.389876	39.839173
9345.20091	Xicheng	Level 2	116.389876	39.839173

A Dataframe with geographical coordinates

Dist2CBD	DistName_En	SubwayDum	wgs_lng	wgs_lat	geometry
9345.20091	Xicheng	Level 2	116.389876	39.839173	POINT (116.38988 39.83917)
9345.20091	Xicheng	Level 2	116.389876	39.839173	POINT (116.38988 39.83917)
9345.20091	Xicheng	Level 2	116.389876	39.839173	POINT (116.38988 39.83917)

A GeoDataframe with a geometry list

- The `geometry` column stores Shapely `Point` objects.

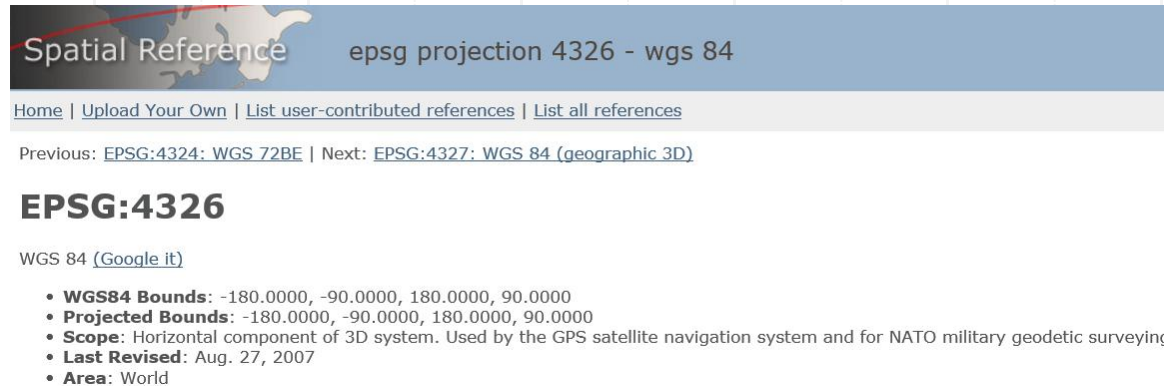


Projection and EPSG

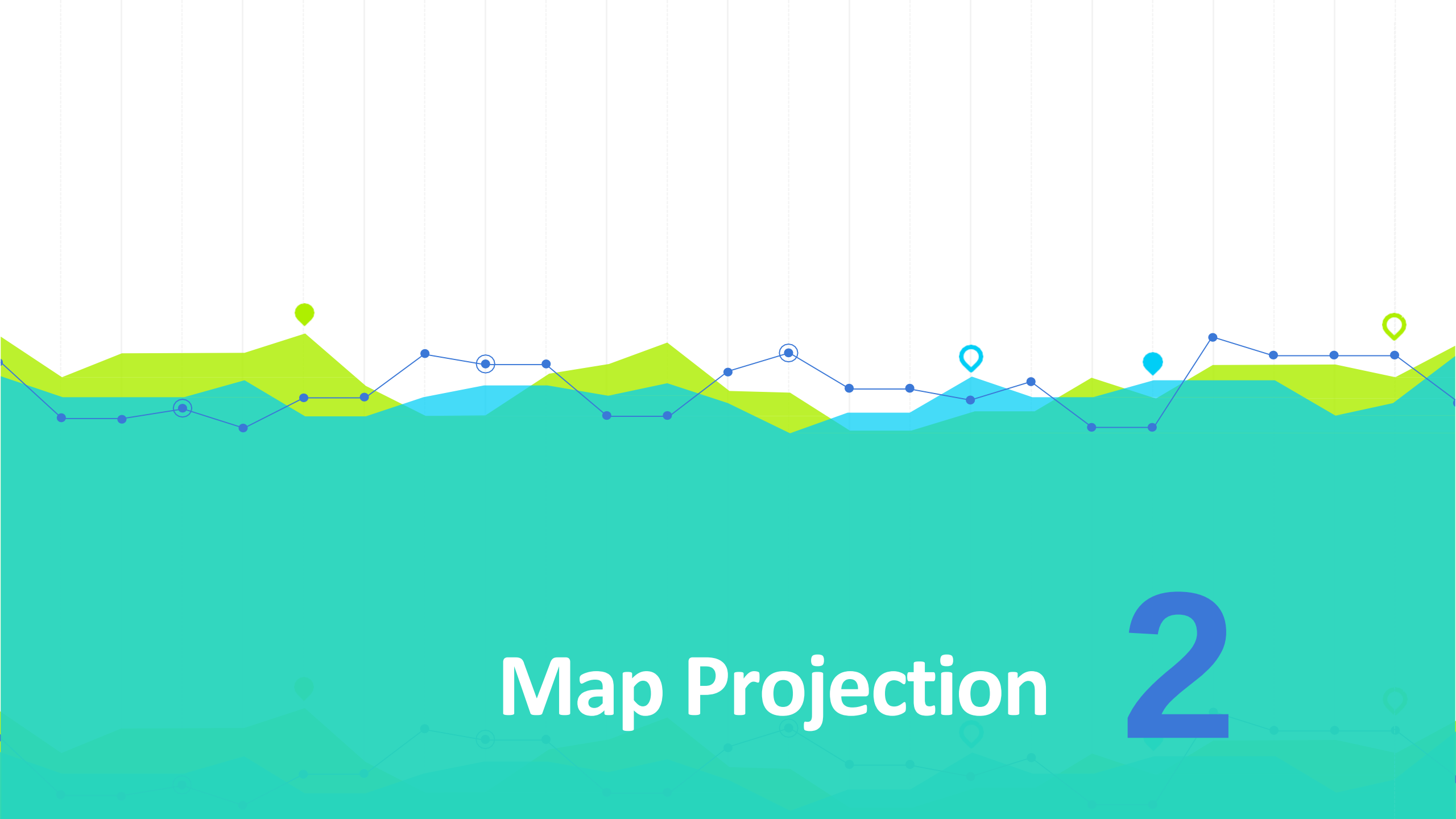
The common way to specify coordinate reference system (crs) is to use the *epsg* code:

- Example: WGS84---[EPSG:4326](#) (longitude and latitude)

```
gdf_combine = gpd.GeoDataFrame(data = df_combine, crs = 'epsg:4326', geometry = geometry_list)
```



- EPSG is a collection of definitions of coordinate reference systems (map projections and datums) and coordinate transformations which may be global, regional, national or local in application.
- Each coordinate system corresponds to a EPSG code.

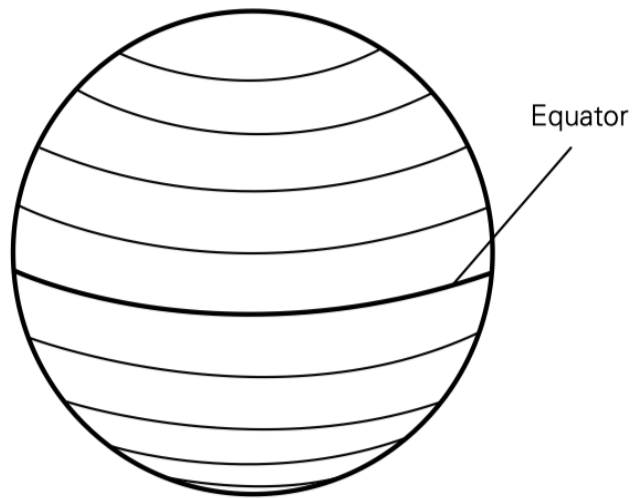


Map Projection

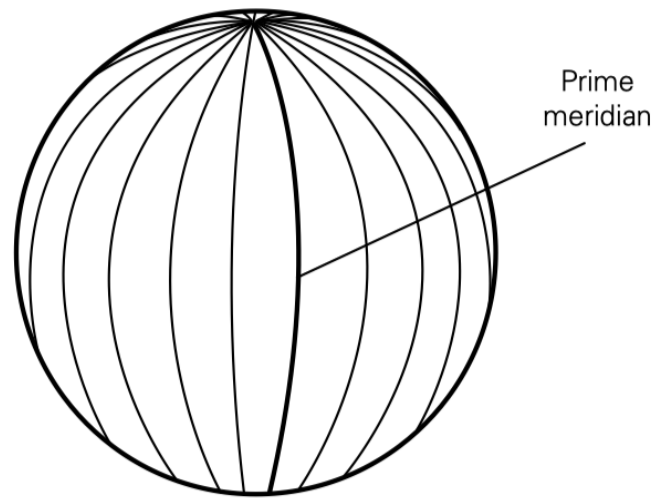
2

What is GCS (geographic coordinate system)?

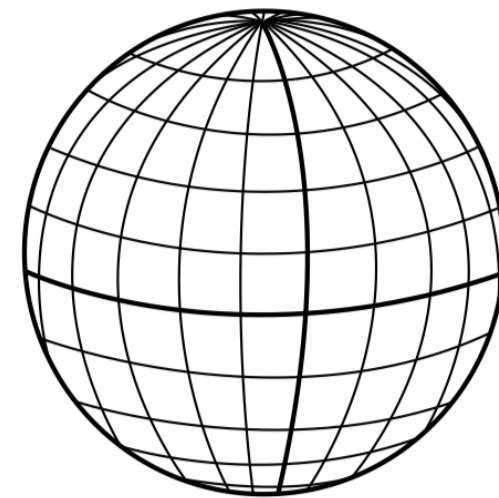
A geographic coordinate system (GCS) is a reference framework that defines the locations of features on a **model** of the earth. It's shaped like a globe—spherical. Its units are angular, usually **degrees**.



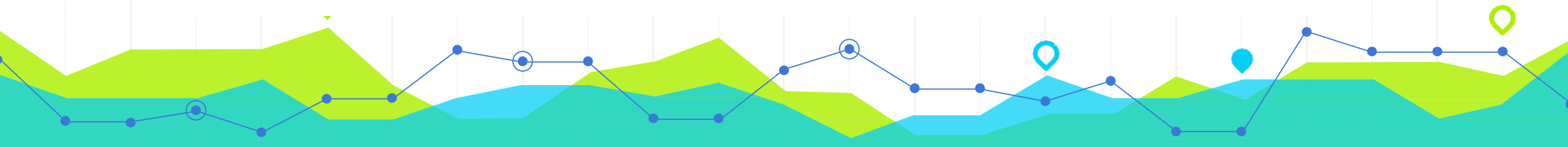
Parallels of latitude



Meridians of longitude

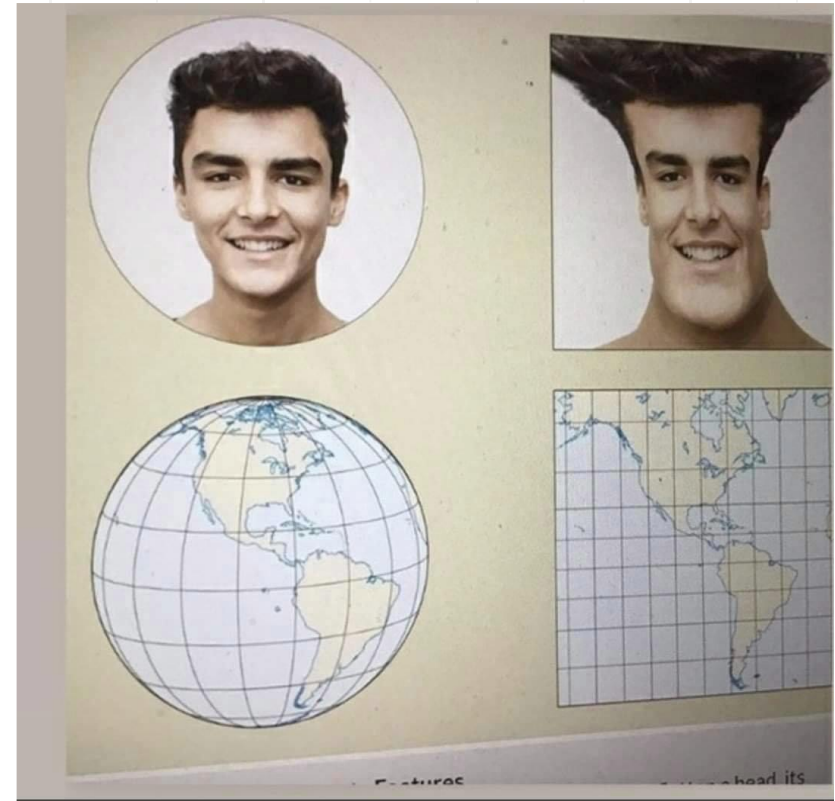


Graticular network



What is PCS (projected coordinate system)?

A **projected coordinate system (PCS)** is flat. It contains a GCS, but it converts that GCS into a flat surface, using math (the projection algorithm) and other parameters. Its units are linear, most commonly in **meters**.



Why all world maps are wrong

Don't be confused

GCS and PCS

What is the relationship and difference between GCS and PCS?

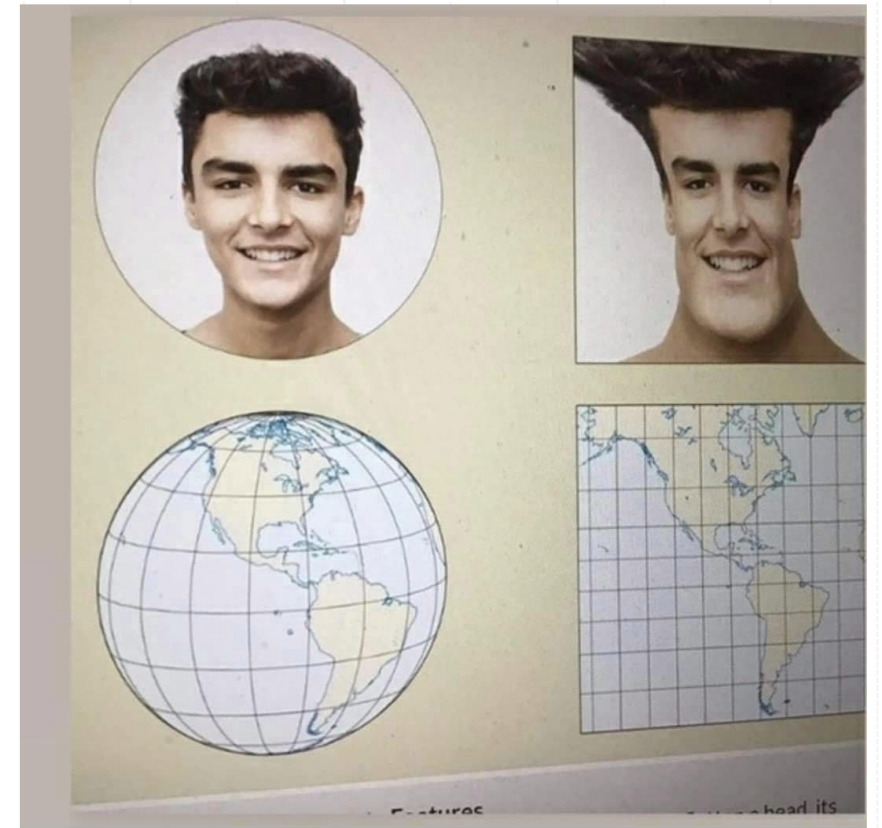
- A GCS defines where the data is located on the earth's surface. In angular units.
- A PCS tells the data how to draw on a flat surface, like on a paper map or a computer screen. In linear units.
- A projected coordinate system (PCS) is a **GCS** that has been flattened using a map projection. A projection is a mathematical algorithm that is used to create a PCS.
- A projection defines a set of parameter values which vary depending on the projection (false easting, central meridian, standard parallel, and so on).
- Your data must have a GCS before it knows where it is on earth. There has to be GCS first, and then a PCS.



Map Projection (key step from GCS to PCS):

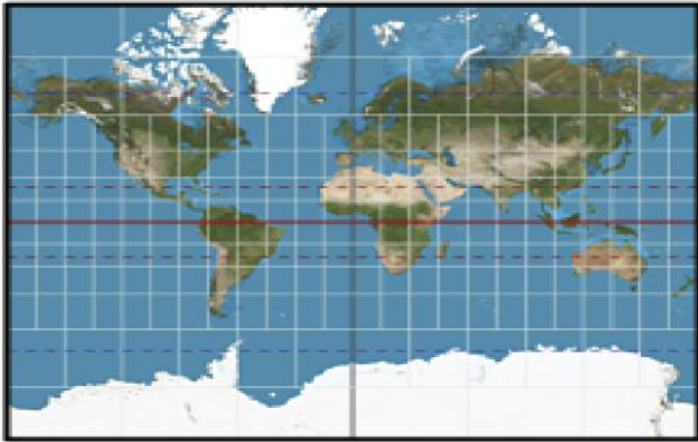
- angular coordinates (e.g., degrees of latitude and longitude) into planar coordinates (e.g., meters, feet).
- All projections distort some combination of either the **shape, size, distance or direction** of land masses.

[Why all world maps are wrong](#)

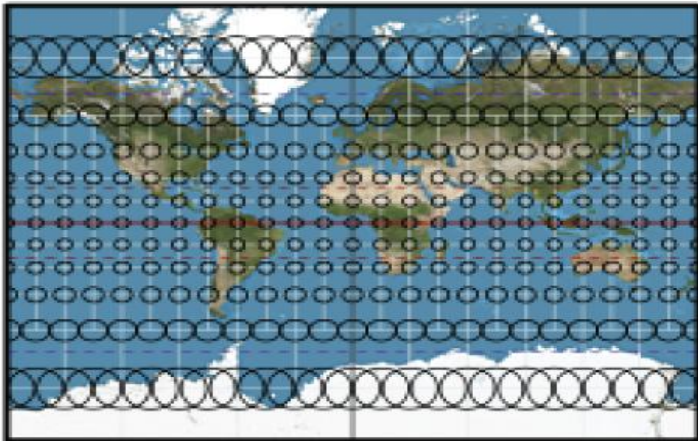


Map Projection – distortion

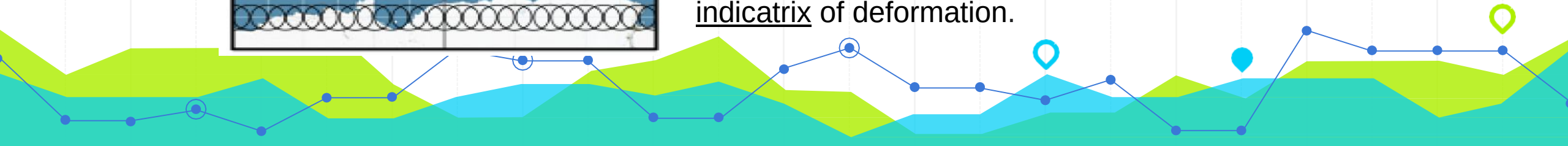
Shape (Conformal projections) - e.g., Mercator projection (Preserves true shape - exaggerates areas)

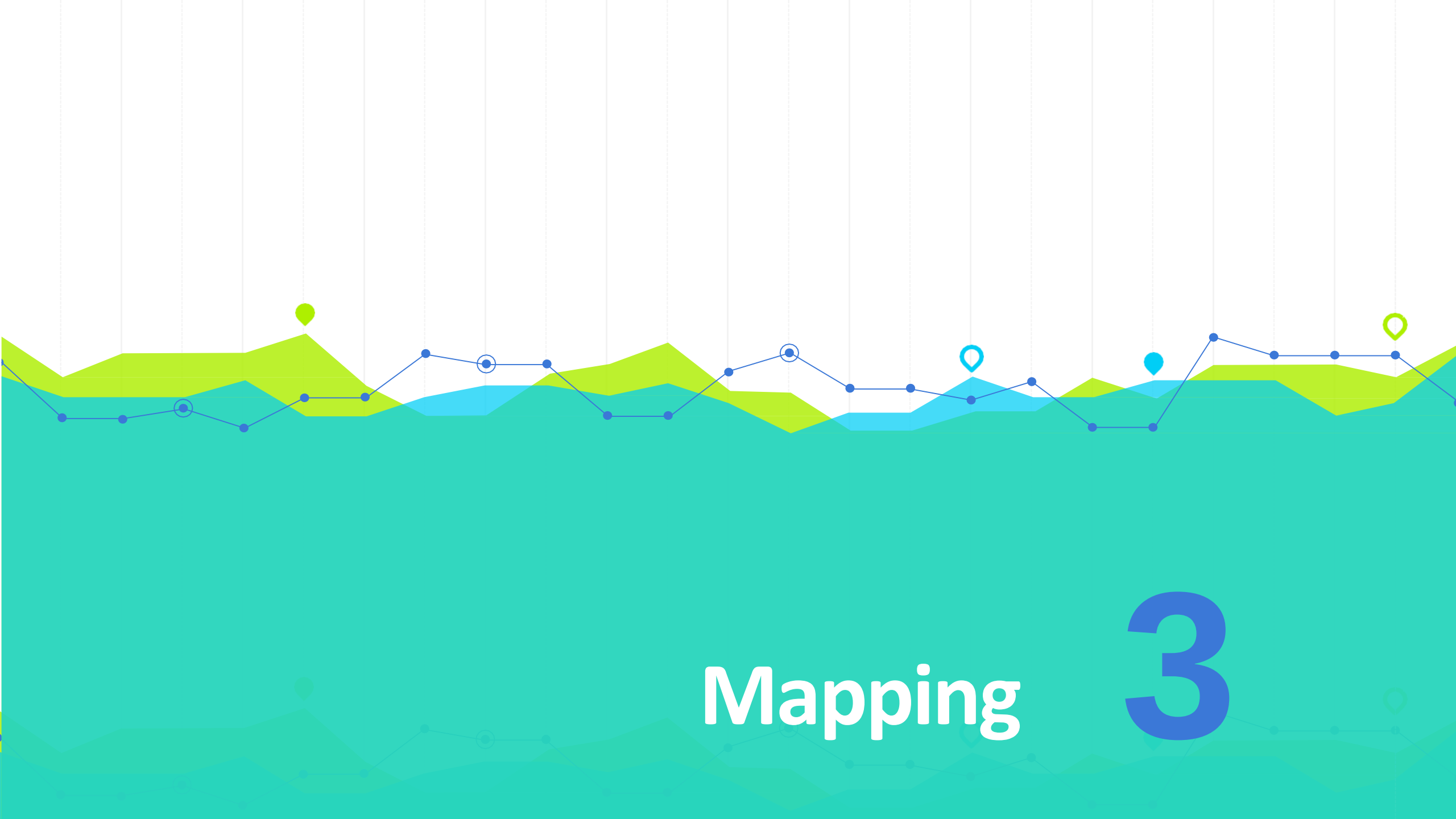


Greenland is portrayed as being larger than Africa, or Australia.



The Mercator projection with Tissot's indicatrix of deformation.

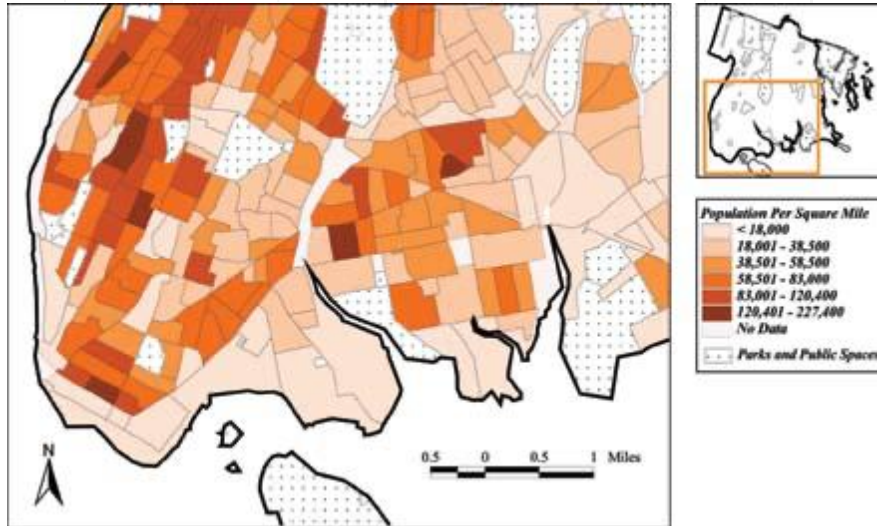




Mapping

3

Data Mapped: Quantitative vs. Qualitative Map



Quantitative maps show numerical relationships of the variables being mapped
Qualitative maps are based on descriptive information, and show location and boundaries of differences of kind or type

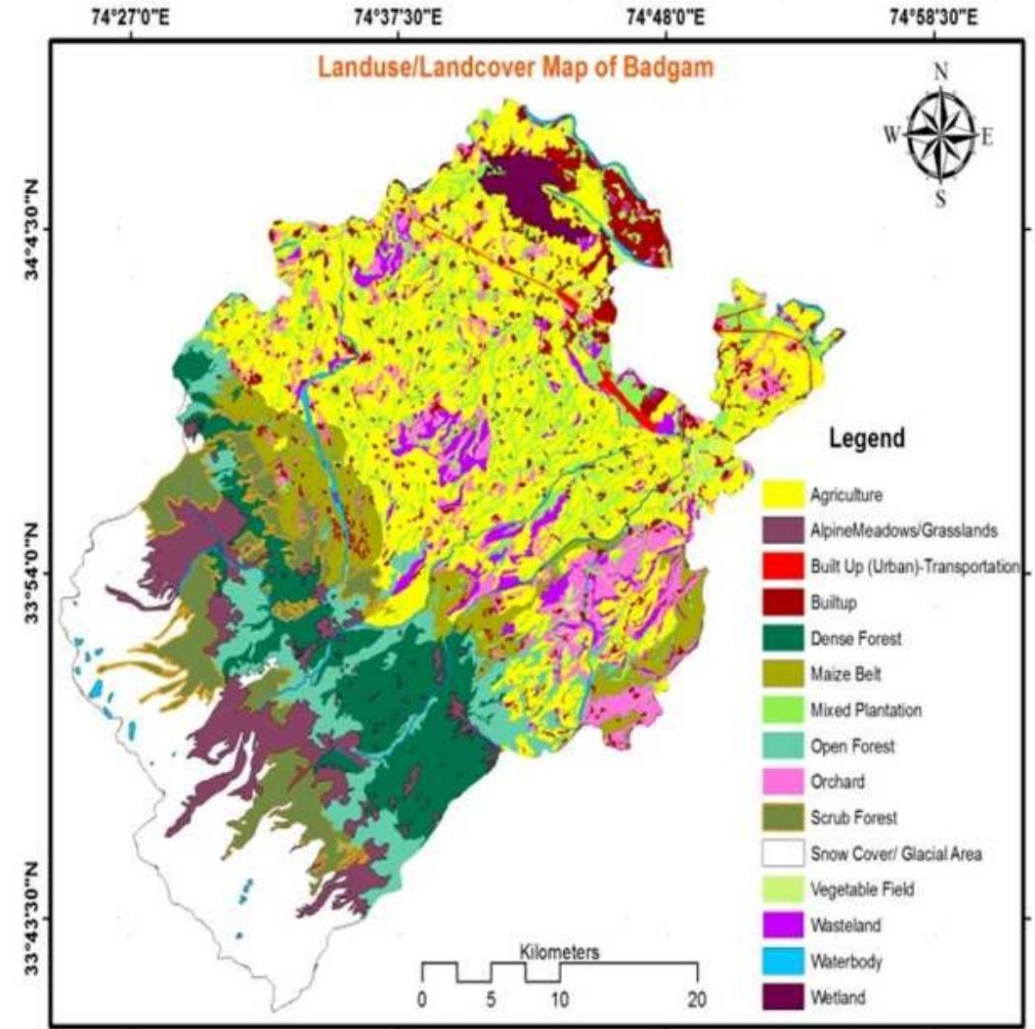
Qualitative maps make use of **nominal** (e.g. land cover class) or **ordinal** (e.g. a ranking) measurement scales, while quantitative maps make use of **interval** (e.g. Celsius temperature) or **ratio** (e.g. density, income, etc..) measurement scales.

Qualitative Thematic Maps — Measurement scales: **Nominal**

Objects are classified into groups. The groups have names, not numeric values. There is no ordering implied. Also called *categorical* or *qualitative* data.

Examples: soils, land use, land cover, zoning, forest types, ethnicity

- The thematic map of a nominal attribute field is often called a “unique value” map.



Qualitative Thematic Maps — Measurement scales: **Ordinal**

Implies a hierarchy of rank—a ranking of classes.

Opinion poll response of “strongly agree,” “agree,” “disagree”

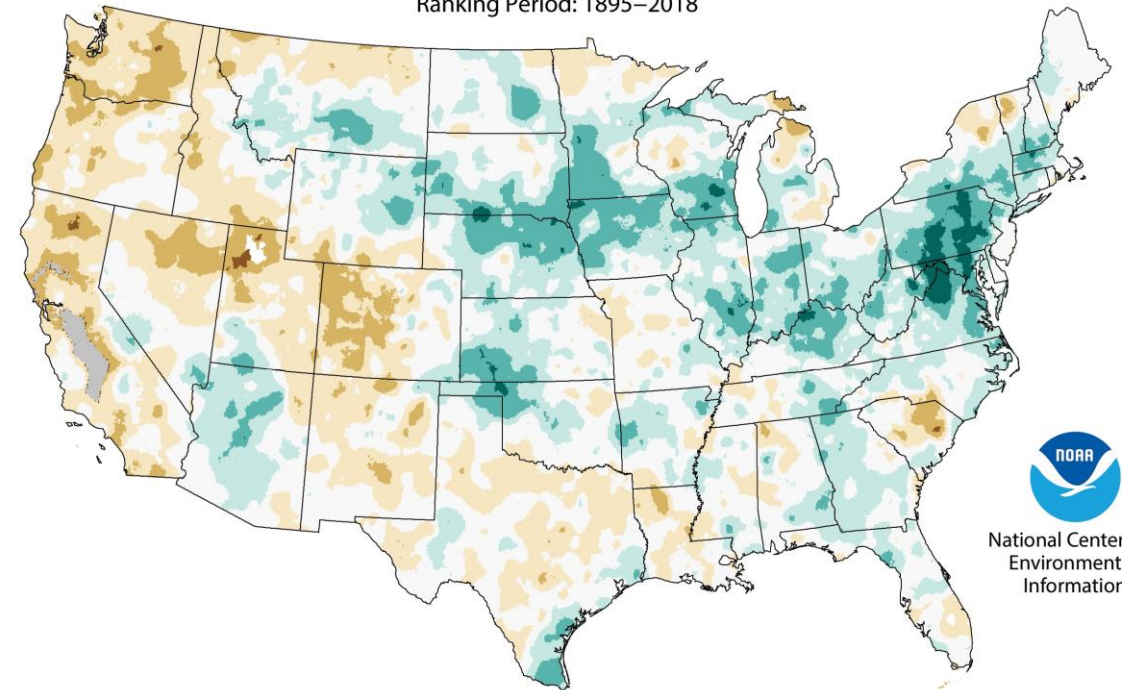
Soils can be ordered from “poorly drained” to “somewhat poorly drained” to “well-drained” to “excessively drained”

*building condition (poor to excellent)
Roads are assigned congestion levels.*

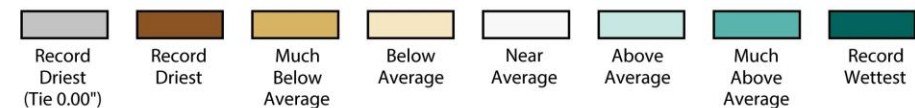
- *We can assign numbers to these categories, but it doesn't automatically imply that we can use arithmetic relationships.*

The map of an ordinal attribute field should use “graduated” symbols or shade patterns.

Total Precipitation Percentiles
June–August 2018
Ranking Period: 1895–2018



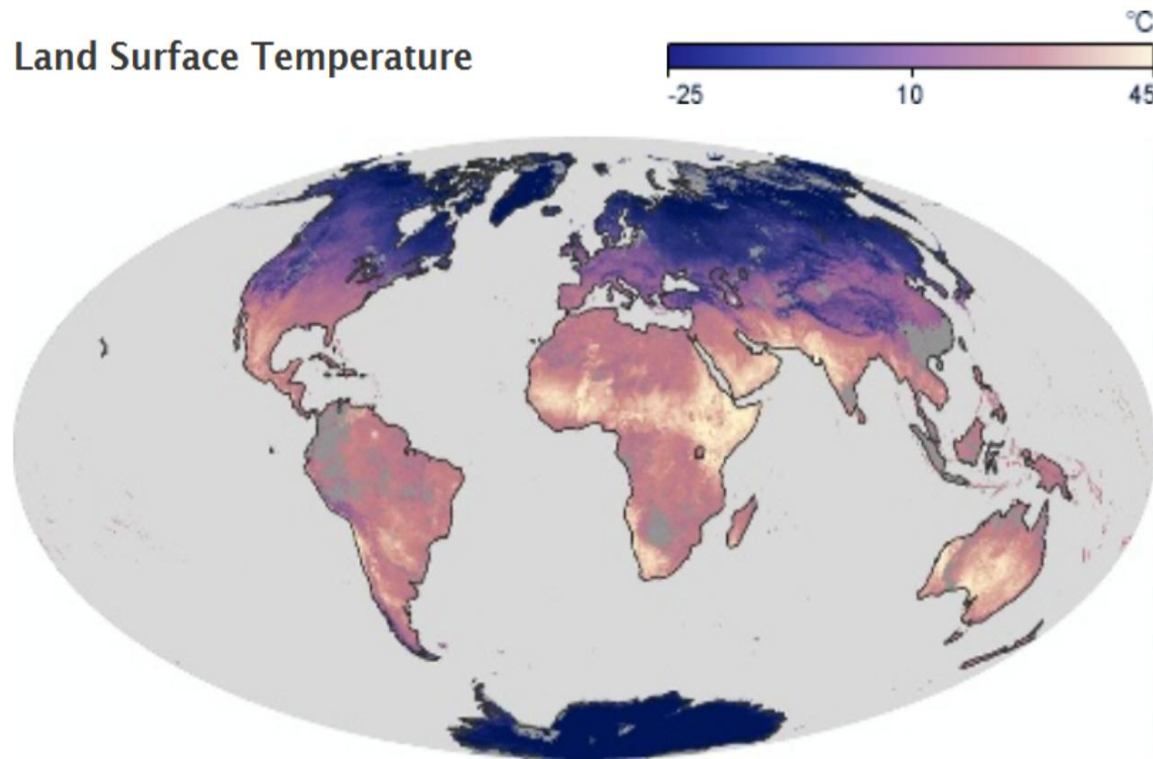
National Centers for
Environmental
Information



Created: Wed Sep 05 2018

Data Source: 5km Gridded Dataset (nClimGrid)

Quantitative Thematic Maps — Measurement scales: **Interval**

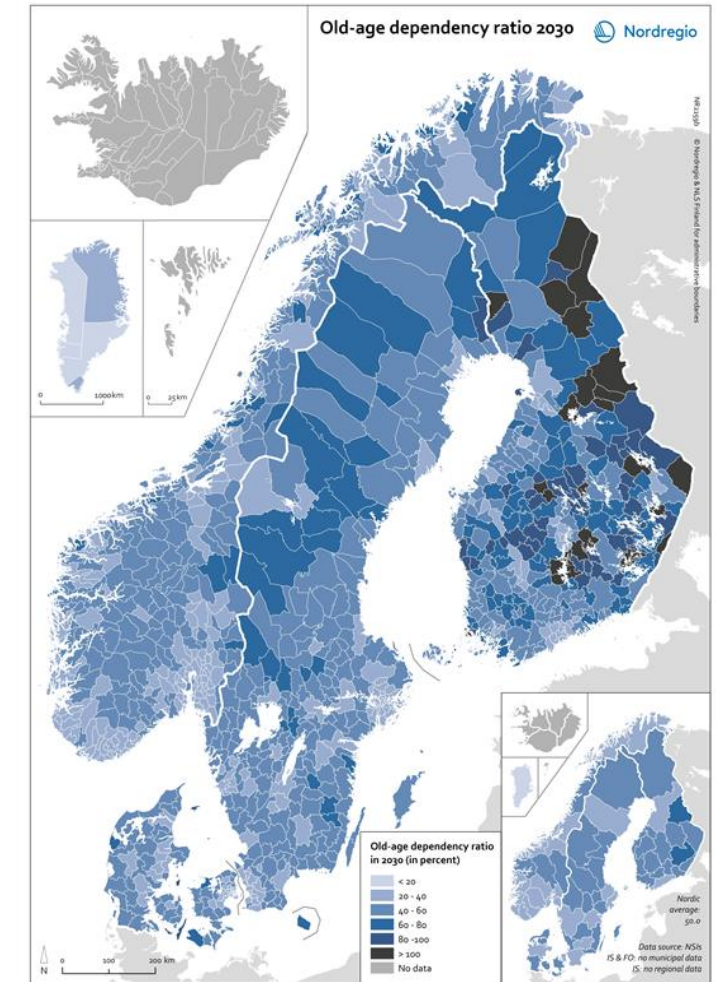


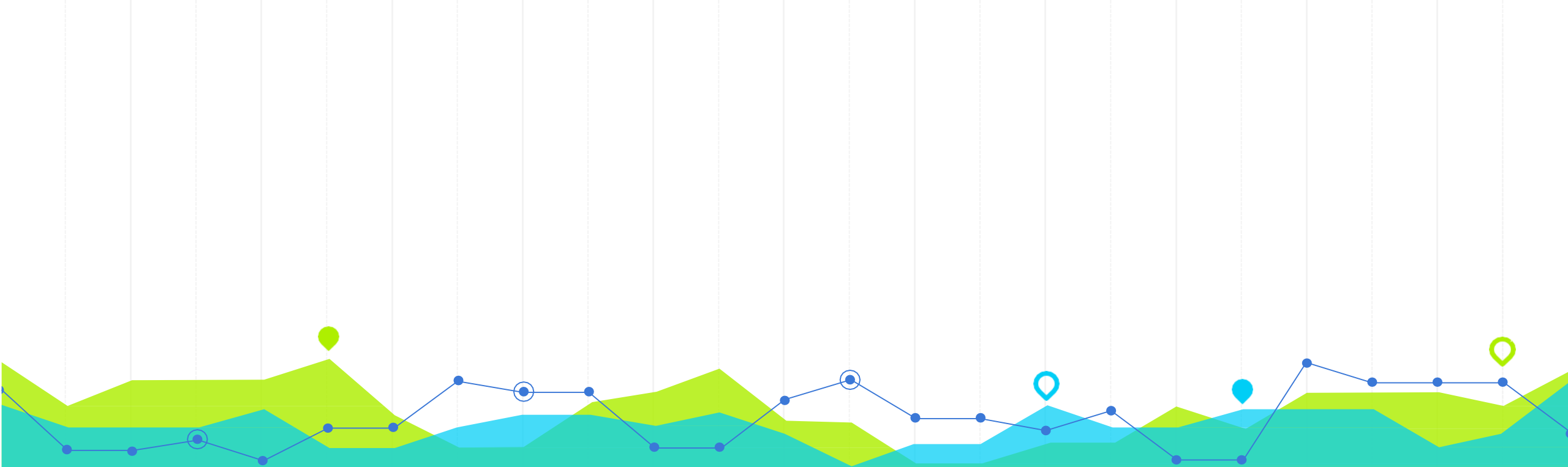
- represent quantitative data
- the difference between values is meaningful and consistent across the scale.
- no true zero point, meaning zero does not indicate the absence of the quantity being measured.
Example: *years, Temperature (in Celsius or Fahrenheit)*
- can't make statements like "twice as much"

Quantitative Thematic Maps — Measurement scales:

Ratio

- Ratio: Ratio–scale magnitudes are absolute, and have a known starting point of zero.
- Ratio, proportion, percentage (proportion x 100)
- These attributes support arithmetic operations (multiplication and division, e.g., "twice as much" or "half as much").
 - Examples: age of structure, square footage of a parcel, assessed value of a parcel of land, road width in feet, air quality measurement.





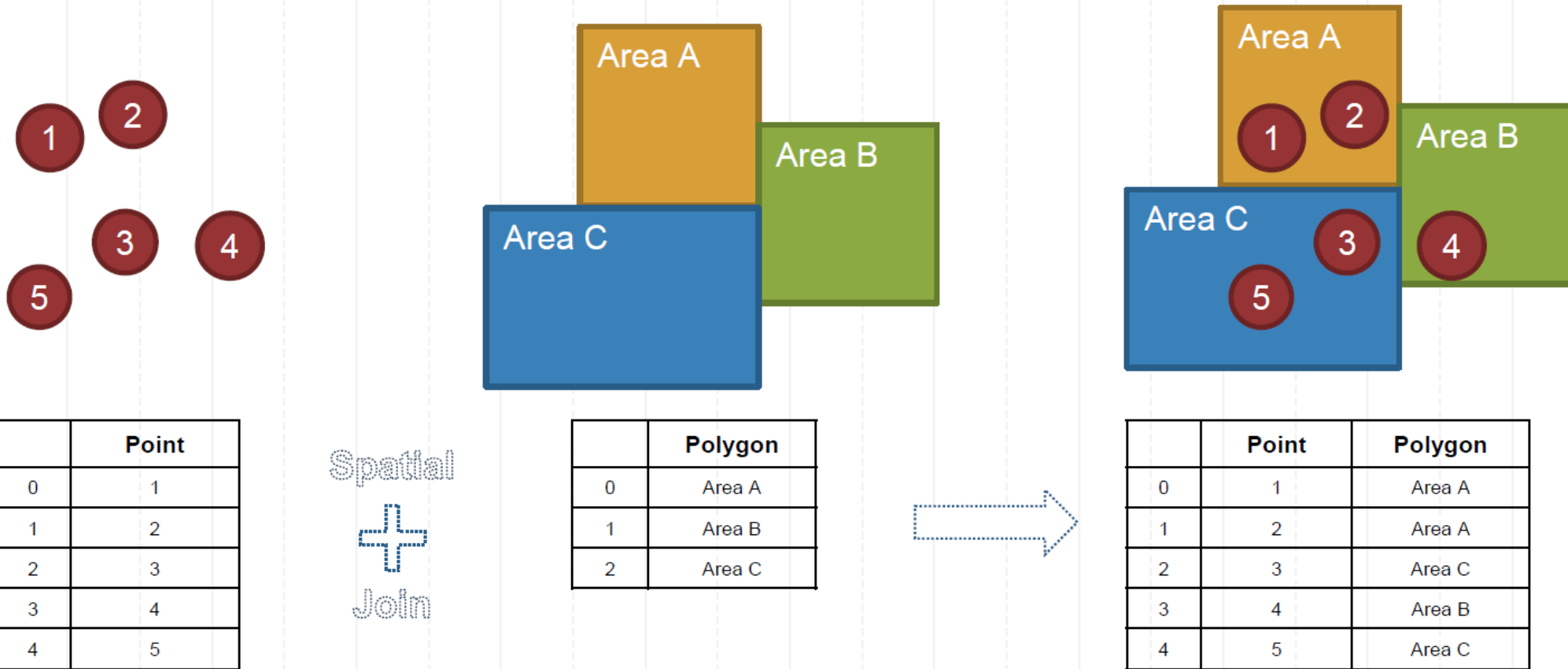
Spatial Operations

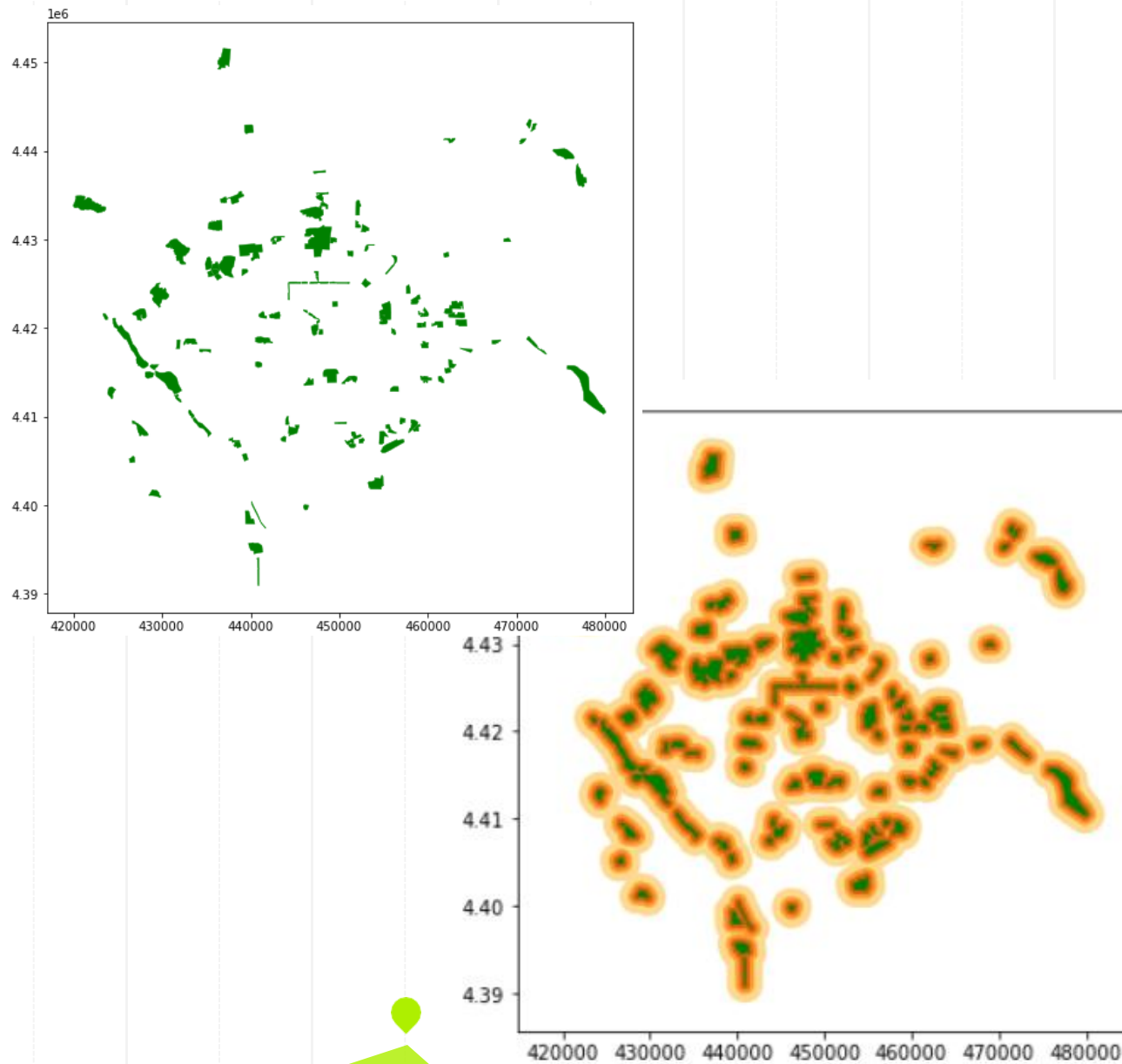
4



Spatial Join

- Two geometry objects are merged based on their **spatial relationship** to one another.





Buffer Analysis

Creating a buffer

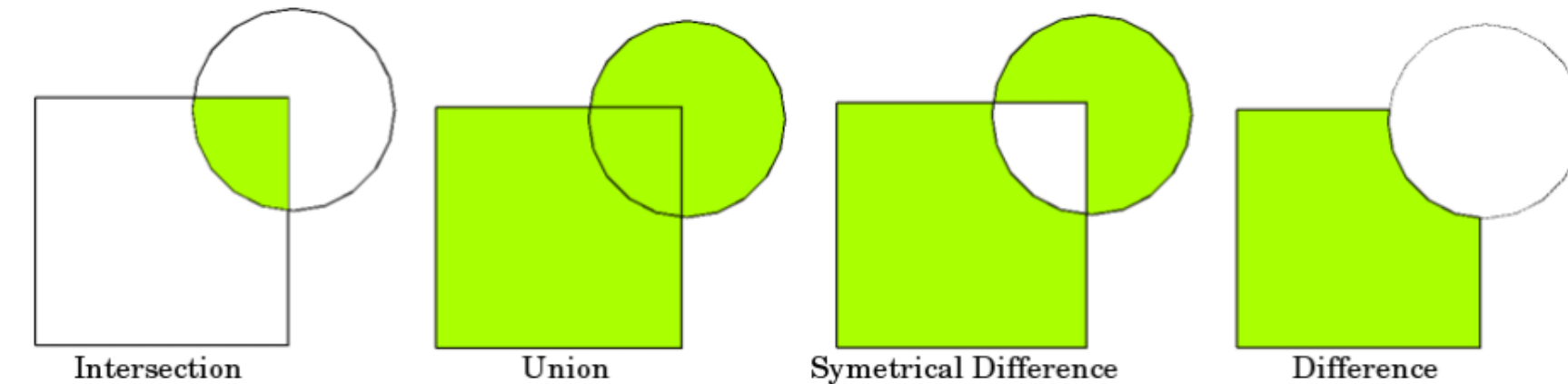
`gdf.buffer()`: It returns a GeoSeries of geometries representing all points within a given distance of each geometric object.

you need to know which crs (projection) you are using to get the correct output you want. That means **if your data is not projected into projection where meters are used, then the output will not be in meters.**

Obtain and plot the 500m, 1000m, and 2000m buffers for each urban park in the city of Beijing.

Overlay

Overlay analyses are GIS operations in which two or more vector layers are combined to produce new geometries. Typical overlay operations include union, intersection, and difference - named after the results of the combination of two layers.



Source: QGIS Documentation



Overlay

Let's say we want to identify those public housing within the 800m threshold and those that are not. We can use the intersection function in `gpd.overlay()`.

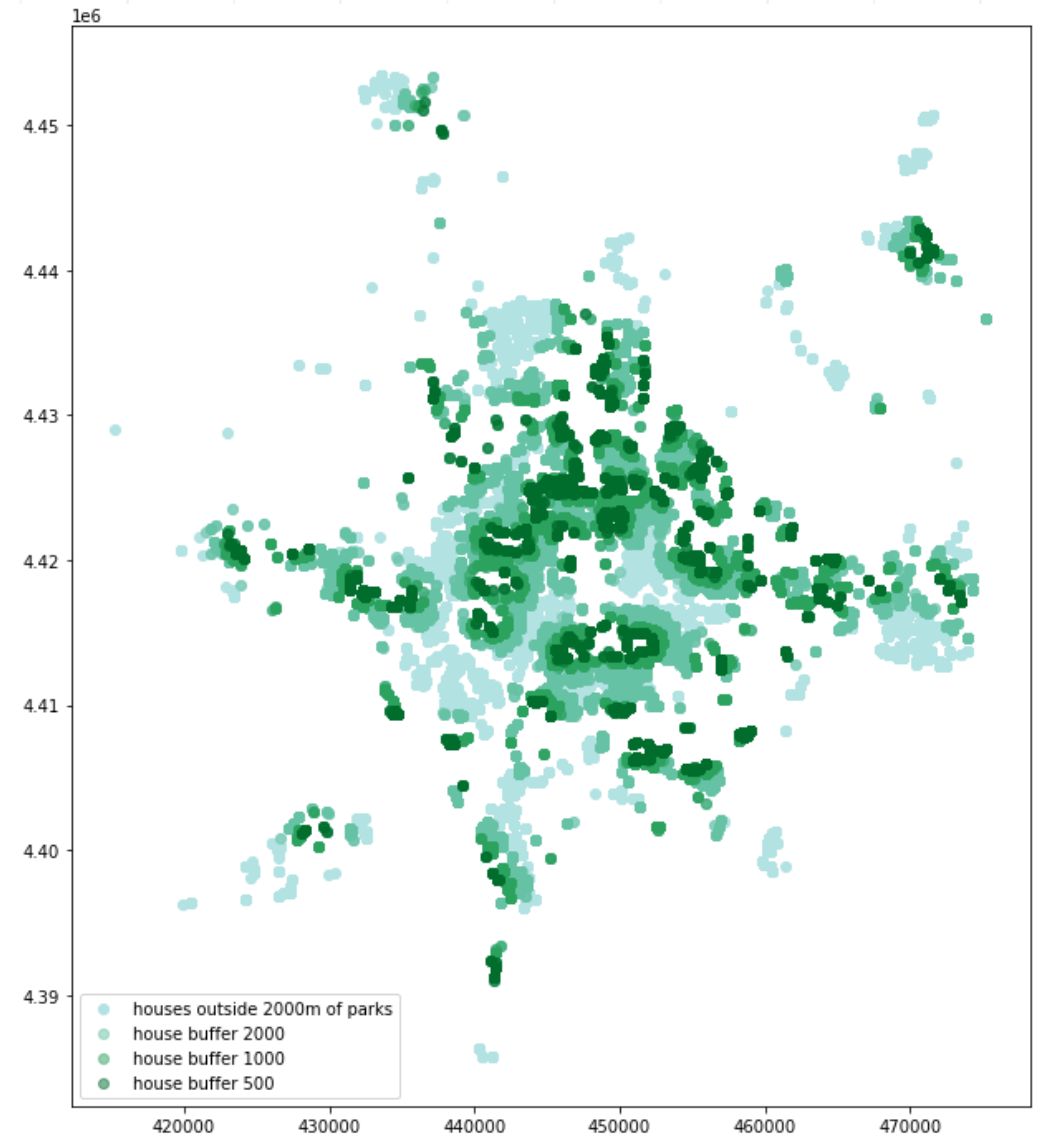
- create 800m buffer area of subway stations using `.buffer()`
- convert public housing from polygon to point using `.centroid()`
- obtain the public housing within the 800m threshold using `intersection` option.
- obtain the public housing outside the 800m threshold using `difference` option.

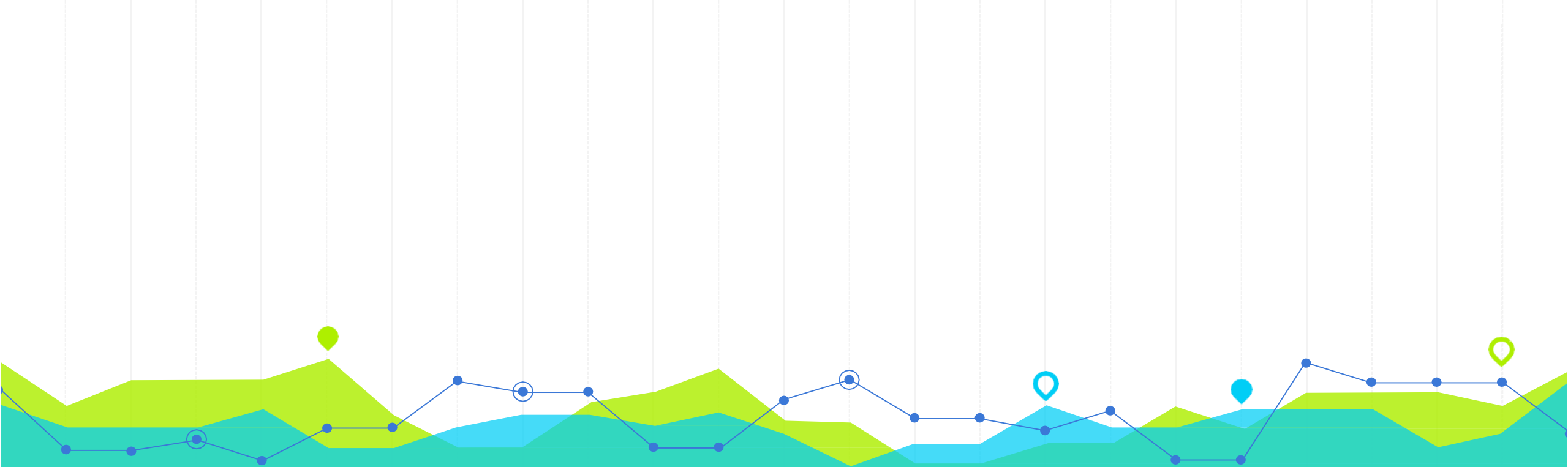


Overlay

Imagine one wishes to identify the houses located within 500m, 1000m, and 2000m, respectively. We can use `gpd.overlay()` to obtain the housing subsets we want.

```
gpd.overlay(gdf_combine_prj, park_500, how='intersection')  
gpd.overlay(gdf_combine_prj, park_1000, how='intersection')  
gpd.overlay(gdf_combine_prj, park_2000, how='intersection')
```





Map APIs

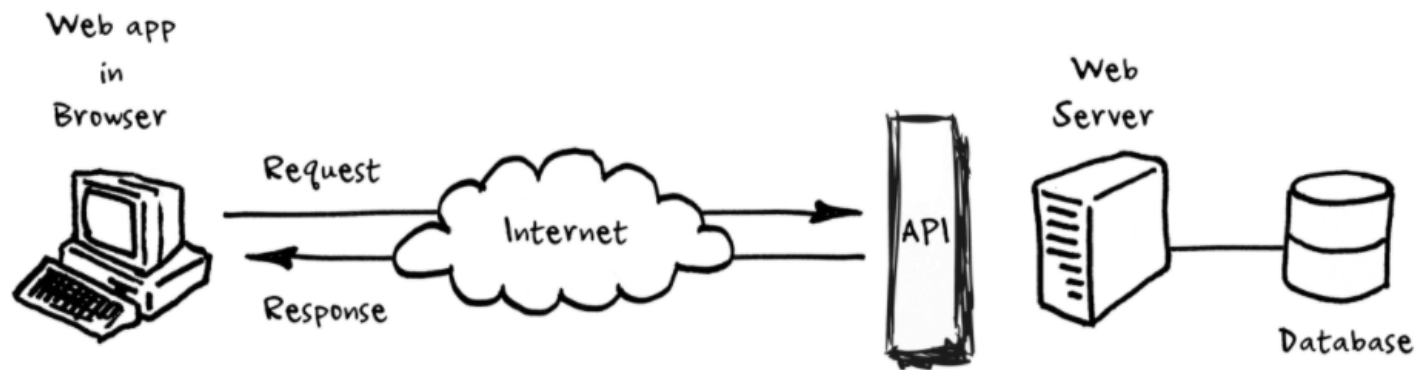
5



What is a web-based API (Application Programming Interface)?

An API allows one program to talk to another program. Many websites or services provide an API so you can query for information or download datasets in an automated way.

- ✓ We send an API a request detailing the information we want.
- ✓ API can return data in response to a request made by a client.



An API is not a database. It is an access point to an app that can access a database.

How API works. Source: [Medium](#)

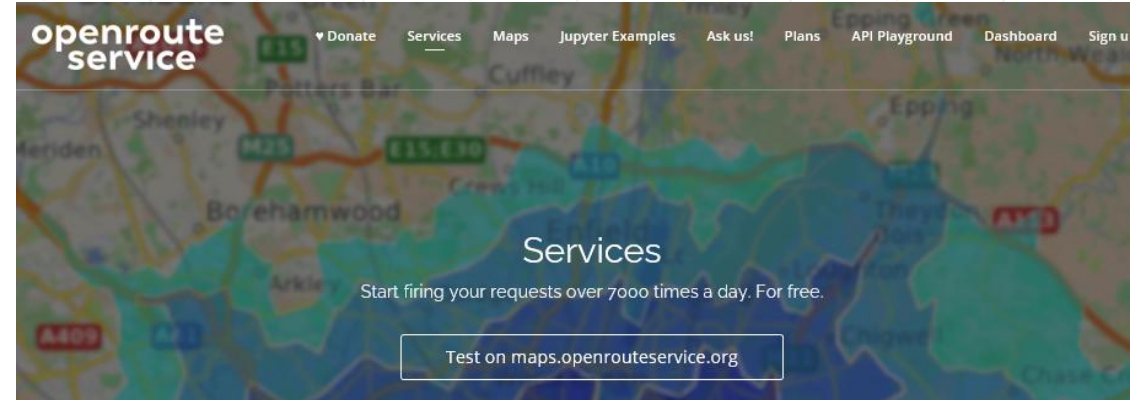
Map APIs

Map APIs for spatial analysis:

- ✓ Getting travel navigation information (directions/distance/duration)
- ✓ Route optimization (shortest distance, least time spent)
- ✓ Geocoding (address to longitude and latitude)
- ✓ Downloading data (obtain spatial entities from OSM and Google MyPlace)
- ✓ Getting real-time weather data
- ✓

Commonly used Map APIs:

- ✓ OpenStreetMap Overpass APIs
- ✓ Google Maps APIs
- ✓ Google Earth Engine API
- ✓ Baidu APIs



User Interface (UI)



Application Programming Interface (API)

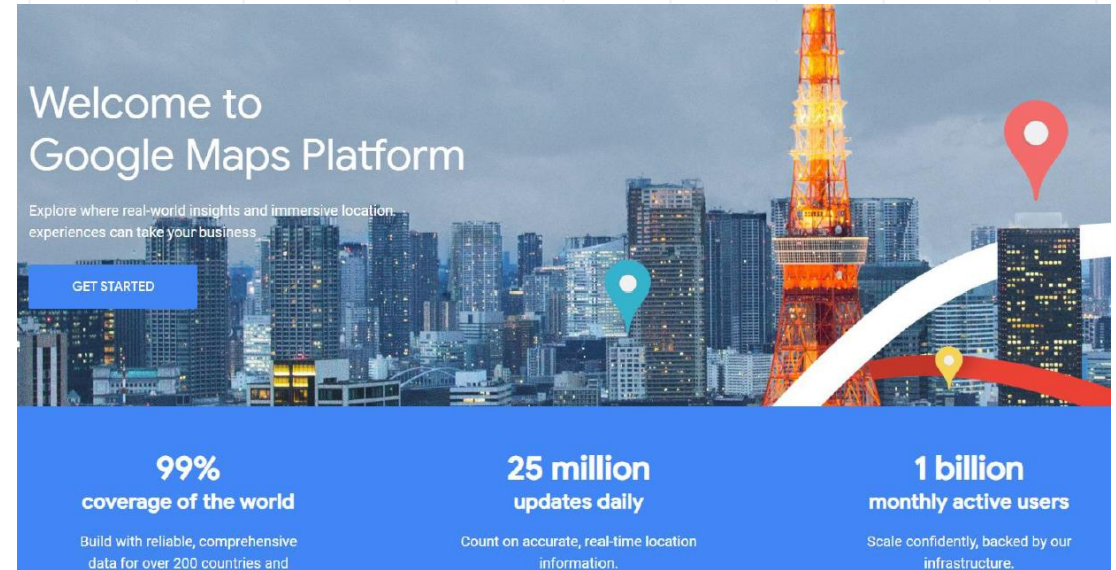


Google Map API (googlemaps library)

The Google Maps library provides us a list of applications using Google Map APIs:

- Directions API
- Distance Matrix API
- Elevation API
- Geocoding API
- Geolocation API
- Time Zone API
- Roads API
- Places API
- Maps Static API

for more: <https://github.com/googlemaps/google-maps-services-python>



Installation of Google Map API:

create a Google API account with an associated API Key. Please get your own API keys following this instruction: <https://developers.google.com/maps/gmp-get-started#api-key>.

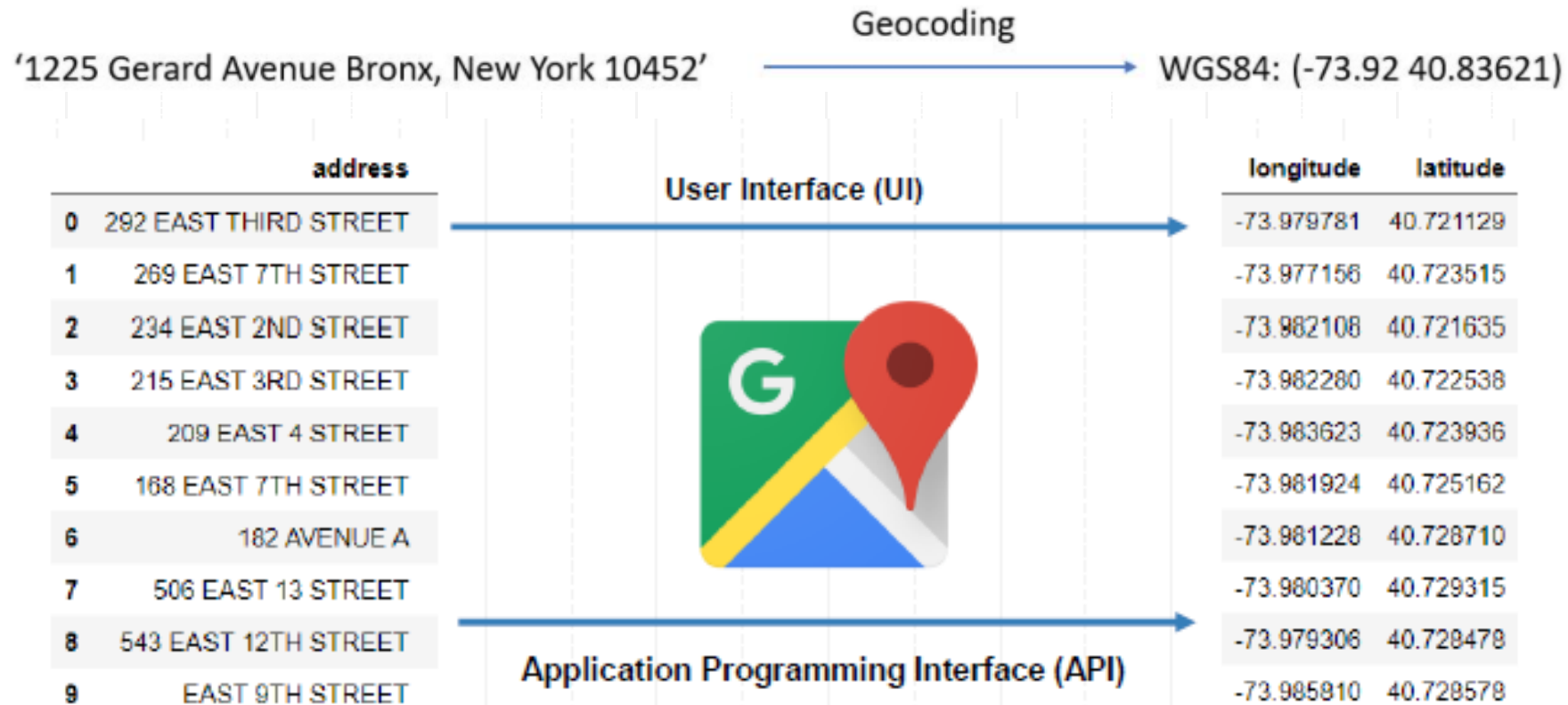
the Python library for Google API is called *googlemaps*. To install the library, we type `!pip install googlemaps`

Once you have created a billing account and project you are eligible for the Google Cloud Platform \$300 free trial and Google Maps Platform recurring \$200 monthly credit. <https://cloud.google.com/maps-platform/pricing/sheet/>. Geocoding API: 0.005 USD per call,

Basic Applications

In this section, we will introduce two basic applications

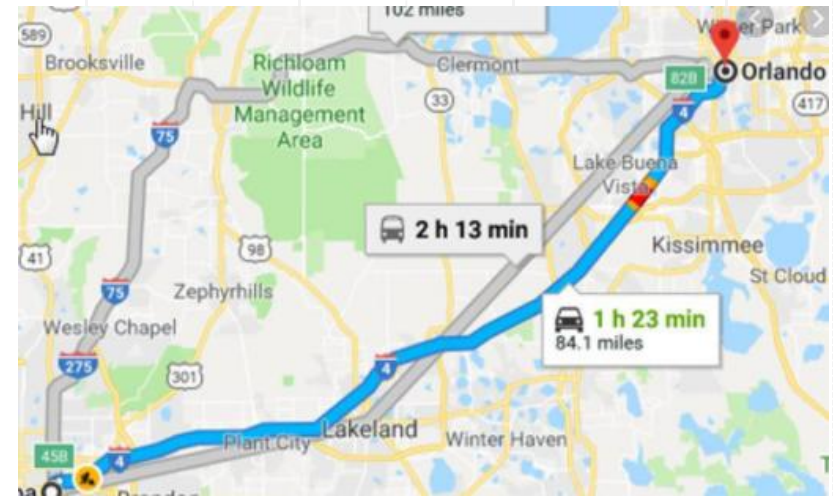
- the **Geocoding API** which returns the corresponding longitude and latitude in terms of the address



Basic Applications

In this section, we will introduce two basic applications

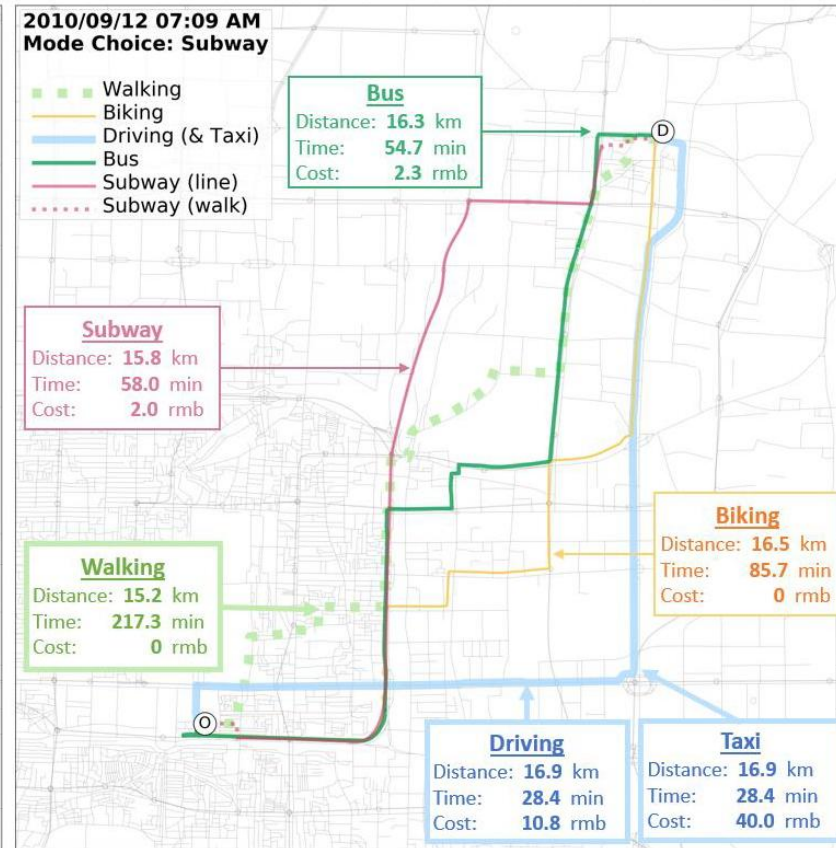
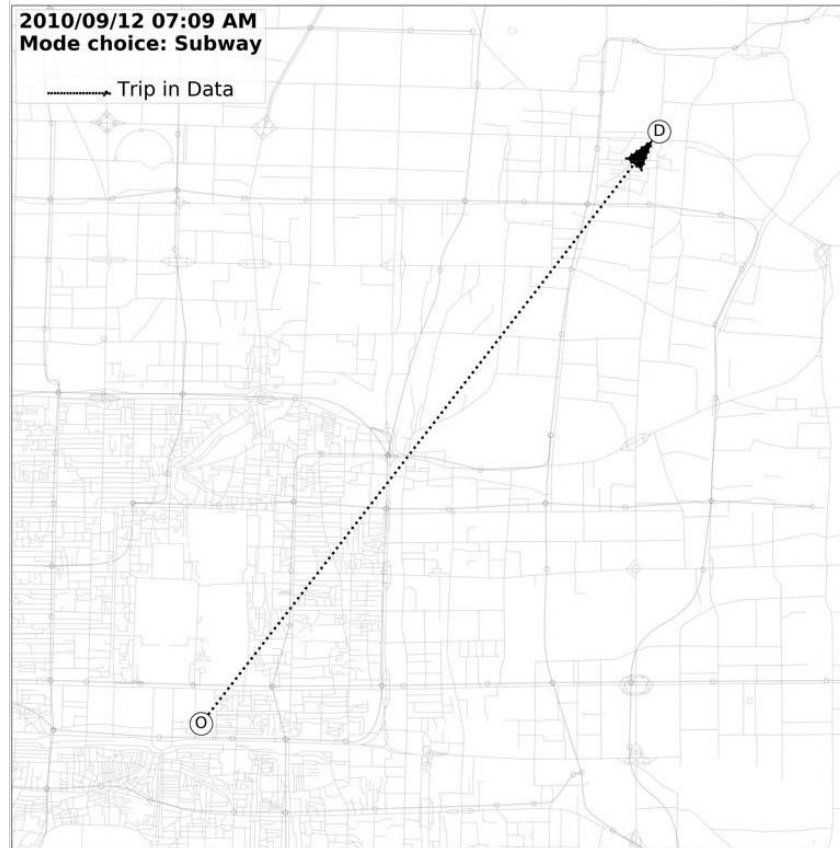
- the **Distance Matrix API** which calculates the travel distance/time from one location to another in terms of travel modes.



	origin	destination	driving distance	driving duration	bicycling distance	bicycling duration
0	New York, NY, USA	Los Angeles, CA, USA	4,494 km	1 day 17 hours	4,720 km	10 days 5 hours
1	New York, NY, USA	Chicago, IL, USA	1,272 km	12 hours 11 mins	1,471 km	3 days 9 hours
2	New York, NY, USA	Houston, TX, USA	2,618 km	1 day 0 hours	2,910 km	6 days 11 hours
3	New York, NY, USA	Phoenix, AZ, USA	3,873 km	1 day 12 hours	4,111 km	8 days 22 hours
4	Los Angeles, CA, USA	Chicago, IL, USA	3,243 km	1 day 5 hours	3,449 km	7 days 5 hours
5	Los Angeles, CA, USA	Houston, TX, USA	2,490 km	22 hours 16 mins	2,582 km	5 days 8 hours
6	Los Angeles, CA, USA	Phoenix, AZ, USA	600 km	5 hours 38 mins	661 km	1 day 10 hours
7	Chicago, IL, USA	Houston, TX, USA	1,744 km	16 hours 13 mins	1,823 km	3 days 20 hours
8	Chicago, IL, USA	Phoenix, AZ, USA	2,822 km	1 day 2 hours	2,838 km	5 days 23 hours
9	Houston, TX, USA	Phoenix, AZ, USA	1,889 km	16 hours 48 mins	1,957 km	4 days 1 hour

More Advanced Applications:

- obtain the travel time, distance and costs based on different travel modes (driving, walking, bus, biking, and walking) for each OD pair

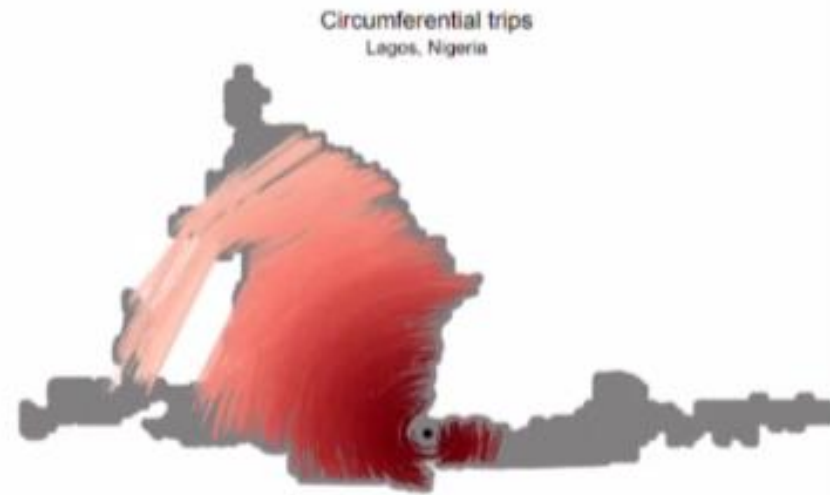
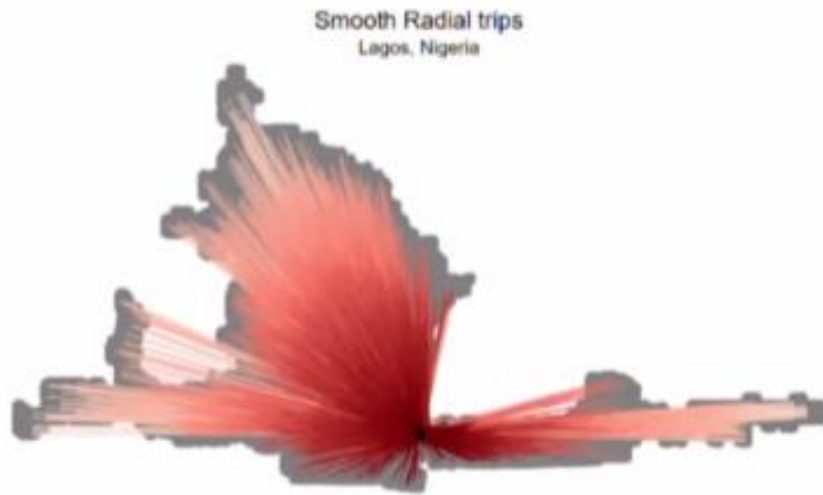


More Advanced Applications:

Akbar et al, 2018:

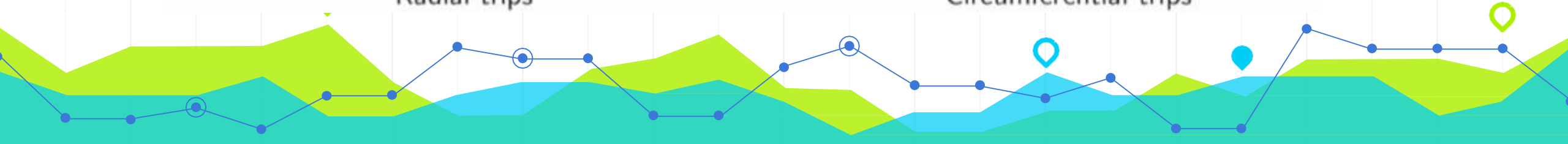
- Obtain POIs from Google MyPlace
- 900M simulated “real time traffic” trips from Google Map
- Compare the real-traffic time/distance with the hypothetical state of no traffic
- Calculate traffic congestion for global cities

Illustration: Lagos in Nigeria



Radial trips

Circumferential trips



Q&A

